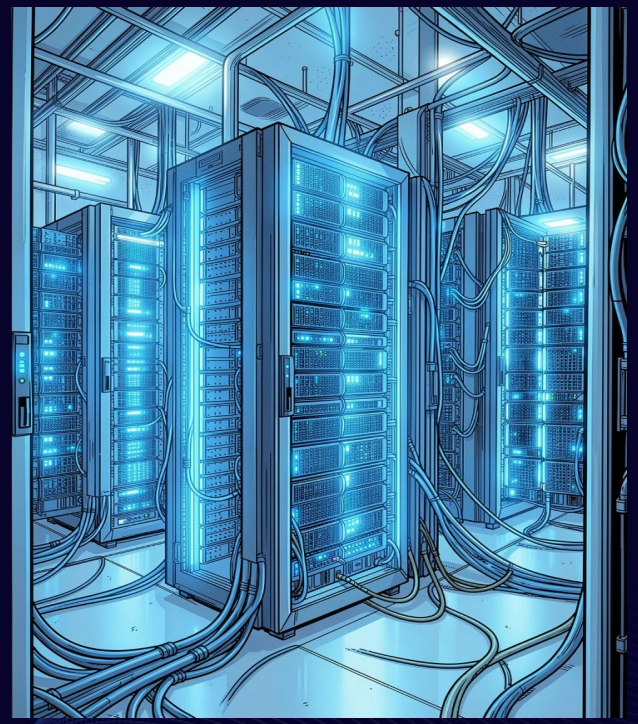

EMBARCADERO CONFERENCE
PATROCINADOR MASTER:
embarcadero
An Idera, Inc. Company

ARQUITETURA DE SOFTWARE

# MONÓLITO OU MICROSSERVIÇOS?



A pergunta não é qual é melhor.
É qual faz sentido para o problema.

Luiz Sfolia • ECON26

**LUIZ SFOLIA**

- Desenvolvedor
- Líder Técnico
- Analista de Sistemas
- Sócio TMR



**TMR**

CONSULTORIA & DESENVOLVIMENTO

**TREINAMENTOS**

**OUTSOURCING**

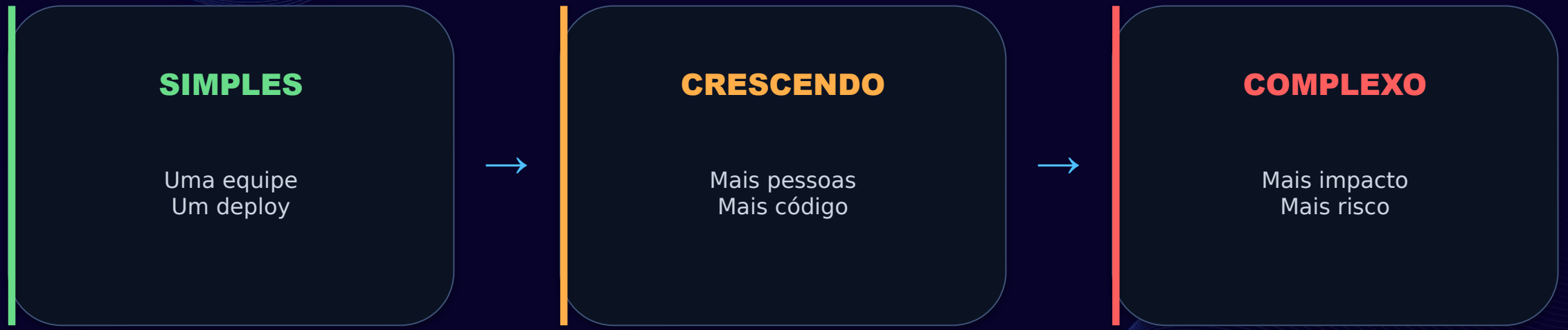
**PROJETOS**

**CONSULTORIA**

O PROBLEMA

# Software começa pequeno

E normalmente funciona muito bem no começo.




**O problema não aparece porque a arquitetura era ruim. Ele aparece porque o contexto mudou.**

COMPARAÇÃO RÁPIDA

# MONÓLITO × MICROSSERVIÇOS

Duas formas de organizar um sistema — cada uma com vantagens e custos.

## MONÓLITO

Uma aplicação principal

### APLICAÇÃO

- Usuários
- Pedidos
- Pagamentos
- Notificações

✓ **Um deploy** • ✓ **Comunicação interna**

✓ **Operação mais simples**

## MICROSSERVIÇOS

Serviços independentes

**Usuários**

**Pedidos**

**Pagamento**

**Notificação**

**API / REDE / MENSAGERIA**

✓ **Deploy independente** • ✓ **Maior autonomia**

**MONÓLITO = SIMPLICIDADE • MICROSSERVIÇOS = AUTONOMIA**

MONÓLITO

# Quando o monólito é a melhor escolha

O monólito não é o vilão.

**Deploy único**

Menos peças para coordenar

**Debug fácil**

Um fluxo mais simples de seguir

**Menor custo**

Infraestrutura mais enxuta

**Equipe pequena**

Menos coordenação entre times

**Simples não significa ruim. Simples significa menos coisas para dar errado.**

SINAIS DE ALERTA

# Quando o monólito começa a doer

O tamanho não é o problema. A dependência é.

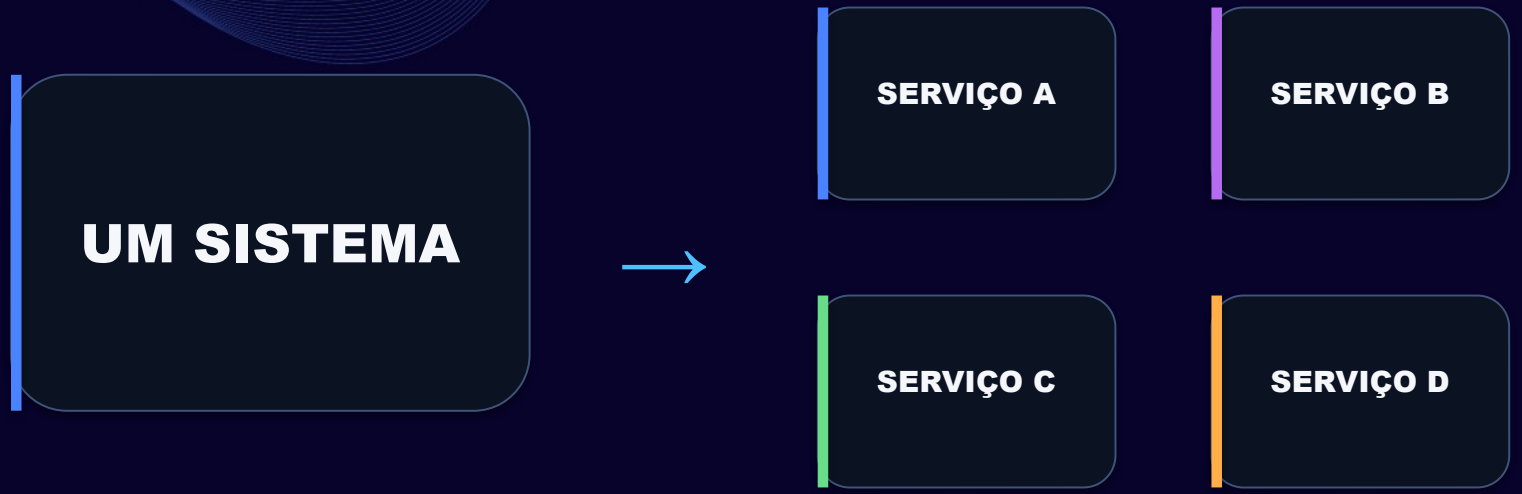
| | |
|---|---|
| **Dependências em cascata** | Uma mudança quebra outras |
| **Builds demorados** | CI/CD começa a virar gargalo |
| **Times bloqueados** | Conflitos e dependências viram rotina |
| **Deploys arriscados** | Cada release vira um evento |

**Quando a arquitetura começa a impedir a evolução, é hora de repensar.**

DIVIDIR PARA CONQUISTAR

# Quando vários sistemas podem ajudar

Separar pode aumentar autonomia — mas também aumenta responsabilidade.




**Mais sistemas, mais conexões**

MICROSSERVIÇOS

# Quando a separação é bem feita

Os ganhos podem ser enormes.

### Times independentes

Evoluem sem se bloquear

### Escalabilidade

Escale só o que precisa

### Deploy separado

Mudanças mais isoladas

### Tecnologias

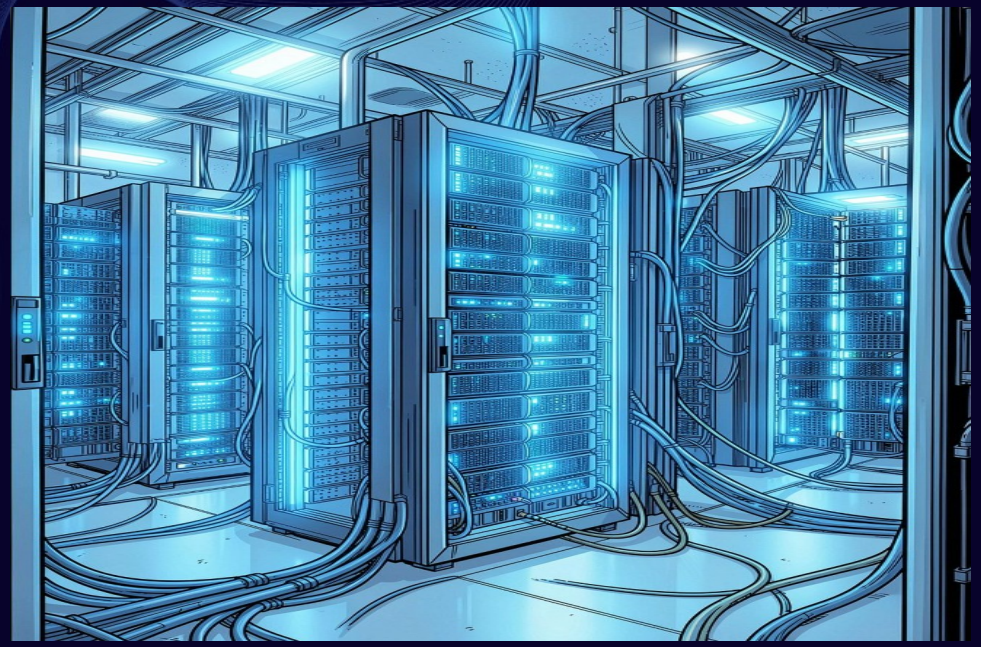
Liberdade com responsabilidade

**Microsserviços não existem para dividir código. Existem para dividir responsabilidades.**

O PREÇO DA DISTRIBUIÇÃO

# Microsserviços não eliminam complexidade

Eles redistribuem a complexidade.

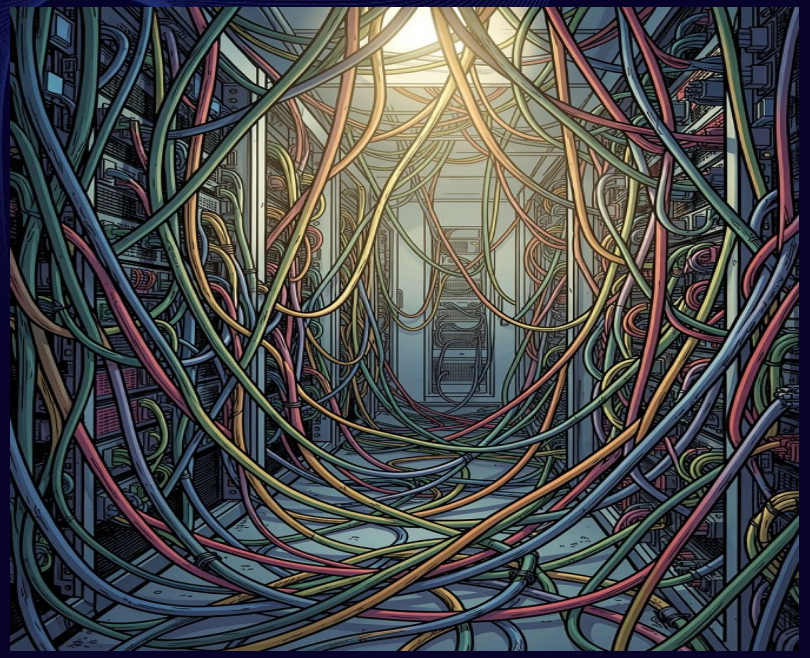


| | |
|---|---|
| **Comunicação** | Latência, falhas e retries |
| **Transações** | Consistência eventual e sagas |
| **Observabilidade** | Logs, métricas e traces |
| **Versionamento** | Contratos entre serviços |

A NOVA VARIÁVEL

# A IA mudou a velocidade de criar software

A barreira para transformar uma ideia em sistema ficou muito menor.



**ANTES**

**Ideia → planejamento → código → sistema**

**HOJE**

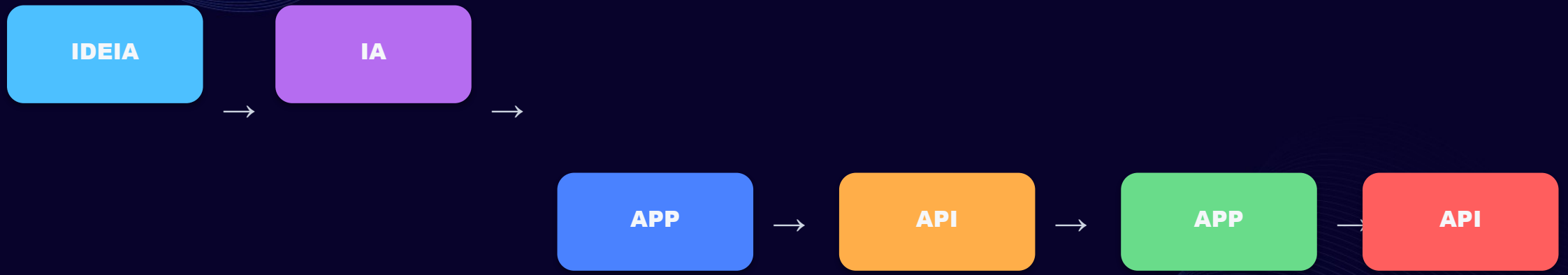
**Ideia + IA → protótipo → sistema**

**Uma ideia pode virar um sistema em horas.**

MAS EXISTE UM EFEITO COLATERAL

# Se ficou fácil criar...

Também ficou fácil criar sistemas demais.




**MAIS SISTEMAS → MAIS CÓDIGO → MAIS DEPENDÊNCIAS → MAIS MANUTENÇÃO**

**Criar ficou barato. Manter continua caro.**

O PONTO MAIS IMPORTANTE

# IA não substitui arquitetura

Gerar código não é a mesma coisa que projetar sistemas.

**A IA PODE**

**Gerar código muito rápido**

**A EQUIPE PRECISA**

**Definir limites e responsabilidades**

**O RESULTADO**

**Velocidade com sustentabilidade**

**A IA diminuiu o custo de criar. Não eliminou o custo da complexidade.**

EVOLUÇÃO SEGURA

# Monólito modular → serviços nos limites certos

A evolução não precisa ser uma reescrita.




**O objetivo não é distribuir tudo. É separar o que precisa ser independente.**

QUER COMEÇAR COM MICROSSERVIÇOS?

# Não comece criando vinte serviços

Comece criando limites.

1. **Modularize o monólito primeiro**
2. **Entenda o domínio**
3. **Separe por responsabilidade**
4. **Planeje dados e contratos**
5. **Prepare observabilidade**

CHECKLIST

# Antes de distribuir, pergunte:

Se a resposta não é clara, talvez ainda seja cedo.

- ✓ **Existe um problema real de autonomia?**
- ✓ **Preciso escalar partes específicas?**
- ✓ **Os limites do domínio estão claros?**
- ✓ **Consigo operar e observar vários serviços?**
- ✓ **O benefício supera o custo da complexidade?**

MIGRAÇÃO SEGURA

# Não reescreva tudo. Evolua por etapas.

A forma mais segura de sair do monólito é reduzir o risco a cada passo.




**A regra prática: extraia primeiro o que precisa ser independente — não o que simplesmente parece fácil de separar.**

ONDE USAR MENSAGERIA

# Mensageria ajuda quando não preciso de resposta imediata

Ela desacopla produtores e consumidores e absorve picos de processamento.

### EVENTOS DE DOMÍNIO

**Pedido criado**
**Pagamento confirmado**
**Cliente atualizado**

### PROCESSAMENTO ASSÍNCRONO

**E-mails**
**Relatórios**
**Imagens / documentos**

### INTEGRAÇÃO ENTRE SISTEMAS

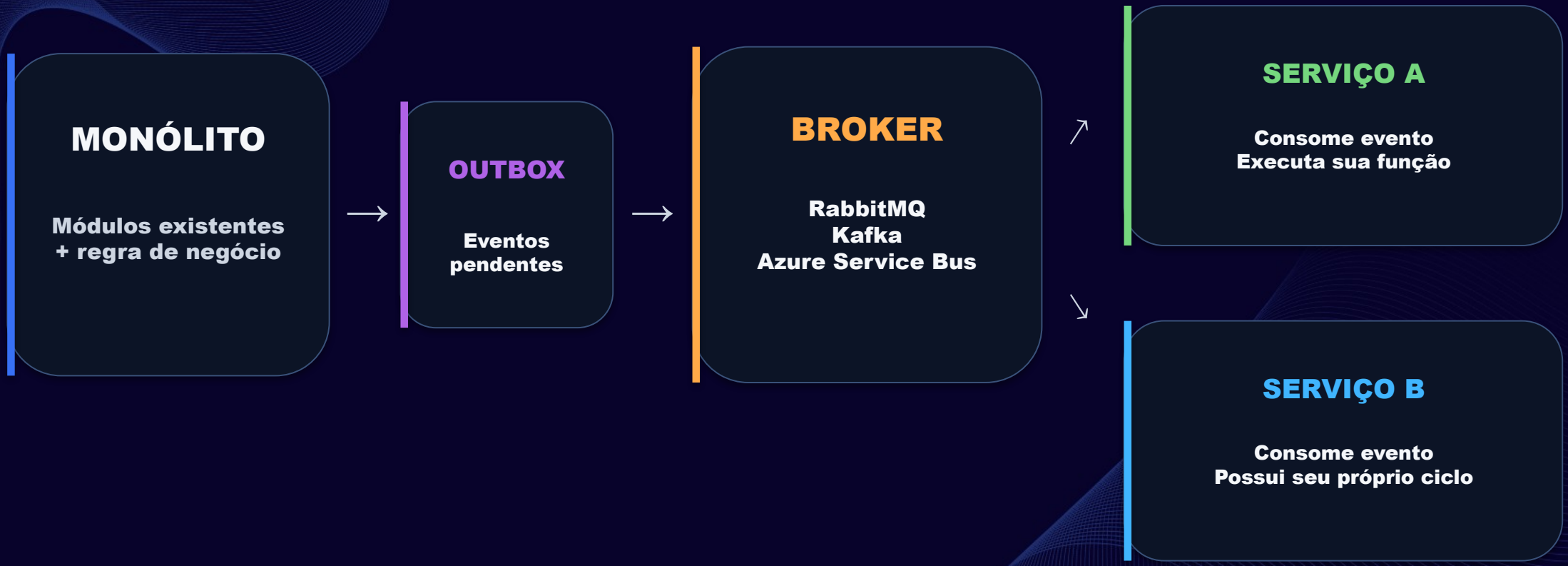
**ERP**
**CRM**
**Estoque**
**Notificações**

**NÃO USE MENSAGERIA APENAS PARA “MICROSSERVIÇAR” UMA CHAMADA SÍNCRONA.**

ARQUITETURA BÁSICA

# Uma transição gradual usando mensageria

O monólito continua funcionando enquanto novos serviços assumem responsabilidades específicas.




**OUTBOX reduz o risco de “gravei no banco, mas falhei ao publicar o evento”.**

PARA ISSO FUNCIONAR NA PRÁTICA

# Mensageria precisa de algumas regras

Sem elas, você apenas troca chamadas HTTP por problemas mais difíceis de enxergar.

### IDEMPOTÊNCIA

O consumidor deve suportar receber a mesma mensagem mais de uma vez.

### RETRY

Falhas temporárias tentam novamente. Falhas persistentes vão para análise.

### CONTRATOS VERSIONADOS

Eventos evoluem sem quebrar consumidores existentes.

### OBSERVABILIDADE

Correlation ID, logs, métricas e tracing para acompanhar o fluxo.

**A arquitetura distribuída fica segura quando as falhas são esperadas — e projetadas.**

CONCLUSÃO

# Nem todo monólito precisa virar microsserviços

E nem todo problema precisa de uma nova arquitetura.

### COMECE SIMPLES

**Estruture bem antes de distribuir.**

### EVOLUA COM PROPÓSITO

**Mude quando houver uma dor real.**

A IDEIA CENTRAL

# O problema não é a arquitetura.

O problema não é o monólito.

O problema não são os microsserviços.

**O problema é usar a arquitetura errada para o momento.**

CASOS REAIS

# Até grandes empresas simplificam

Simplificar pode ser uma decisão arquitetural madura.

**Segment**

**140+ → 1**

**Istio**

**Menos componentes**

**Prime Video**

**90% ↓ custo**

**A lição não é “volte para o monólito”. É medir o custo da complexidade.**

CASO: SEGMENT

# Muitos serviços

Quando a complexidade operacional começa a custar mais do que entrega.




**ANTES**

**140+**

**SERVIÇOS**

→

**DEPOIS**

**1**

**SISTEMA CONSOLIDADO**

**Mais componentes podem significar mais deploys, mais dependências e mais pontos de falha.**

https://www.twilio.com/en-us/blog/developers/best-practices/goodbye-microservices

CASO: ISTIO

# Simplificar também é arquitetura

Consolidar componentes pode reduzir a complexidade operacional.




**ANTES**

**Mais componentes para coordenar**

→

**DEPOIS**

**Menos peças, mais simplicidade**

https://research.google/pubs/the-monolith-strikes-back-why-istio-migrated-from-microservices-to-a-monolithic-architecture/

CASO: PRIME VIDEO

# Distribuir também tem custo

A arquitetura precisa entregar mais valor do que custa.




**ANTES**

**Componentes conversando pela rede**

**DEPOIS**

**Componentes mais próximos e simples**

## 90% ↓ de custo

https://www.linkedin.com/pulse/do-serverless-ao-monolito-obtendo-redu%C3%A7%C3%A3o-de-90-dos-custos-sakamoto/

A MELHOR ARQUITETURA

# Resolve o problema de hoje sem dificultar o amanhã.

**Arquitetura é uma decisão de negócio tanto quanto técnica.**

**Não escolha por moda. Escolha pelo contexto.**

# OBRIGADO!

# EMBARCADERO CONFERENCE 26

PATROCINADOR MASTER: embarcadero An Idera, Inc. Company

**Quer me ver na #ECON27? Acesse o QRCode e avalie minha palestra!**


## Luiz Sfolia

@luizsfolia

luizsfolia

Luiz.sfolia@tmrti.com.br

(51) 9 9822-2731


TMR CONSULTORIA & DESENVOLVIMENTO